Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Calling functions inside loops

Today
Welcome to your next lesson: Calling functions inside loops.

I'm getting your personalized lesson ready—based on what you've learned so far, what's ahead, and the best ways to help you practice.

Step by step, you're moving forward.

Imagine you have a list of prices and you need to add sales tax to each one 🧾

You could write the tax formula over and over inside the loop, or you can write it once in a helper function and let the loop call that function for every price.

Python

```
def add_tax(price):
    return price * 1.08

prices = [10, 20, 30]
for price in prices:
    final_price = add_tax(price)
    print(final_price)
```

Here the loop just says: "for each price, ask add_tax what the final value is."

Quick check-in for you, Ron:

Does this idea make sense so far: loop = goes through the list, helper function = does the per-item work?

You can answer with: "makes sense" or "not yet" and I'll adjust.

Makes sense

Nice 👍

Let's zoom in on the helper function part only.

In this code:

Python

```
def add_tax(price):
    return price * 1.08
```

Step by step:

def add_tax(price): → you define a helper that knows how to handle one price.

return price * 1.08 → it calculates and gives back the new value for that one price.

So this function's only job is: "given a single price, tell me the price with tax."

Next small step:

Can you picture what add_tax(50) would give back?

You don't need to calculate exactly, just say: "it would return 50 times what?"

54

Perfect

It returns 50 * 1.08, which is 54.0 — you've got it.

Now let's connect the function with the loop.

Visually, the flow is:

prices list → for loop → add_tax (one item) → result

In code:

Python

```
def add_tax(price):
    return price * 1.08

prices = [10, 20, 30]

for price in prices:
    final_price = add_tax(price)   # helper is called here
    print(final_price)
```

For each step of the loop:

price becomes one item from the list
add_tax(price) does the work for that one item
we use the result (here we print it)
Before we do an exercise, quick yes/no:

Are you okay with how the loop is calling the helper function on each item?

Yes, I'm okay

Great ☑

Let's do a tiny real-world style task using the same pattern.

Say you have this list:

Python

```
scores = [70, 85, 90, 100]
```

You want a helper that adds 5 points to a single score, and a loop that uses that helper for every score.

The pattern will be:

Define helper: “given one score, add 5 and return it”.
Loop over scores.
Inside the loop, call the helper with the current score.
Your turn 🚀

In the code editor, try this:

Write a function boost_score(score) that returns the score plus 5.
Make a list scores = [70, 85, 90, 100].
Write a for loop that:
goes through scores
calls boost_score for each score
prints the boosted score

Run it first, then Submit code so I can check.

Python

```
scores = [80, 85, 90, 100]
def boost_score(score):
    return score + 5

for score in scores:
    print(f"{boost_score(score)}")
```

Nice work, Ron 🎯

You:

Defined a clear helper:
Python

```
def boost_score(score):
    return score + 5
```

Looped over the list and delegated the per-item work to the helper:
Python

```
for score in scores:
    print(f"{boost_score(score)}")
```

That’s exactly the “call a function inside a loop” pattern this lesson is about.

Next small idea: instead of just printing boosted scores, we can build a new list of them using the helper.

Visually:

original list → loop → helper → new list

Would you like to try that next with something fun like a list of names (e.g., turning "Ron" into "Ron!"), or stick with numbers one more time?

Let's do names!

Love it 😄👀

We’ll use the same pattern, just with strings instead of numbers.

Goal:

Start with: names = ["Ron", "Sam", "Alex"]
Helper function handles one name and returns it with an exclamation mark, like "Ron!"
Loop goes through names, calls the helper for each name, and builds a new list like ["Ron!", "Sam!", "Alex!"]

Visually:

names → for loop → excited(one_name) → excited_names

Your turn in the code editor:

Create names = ["Ron", "Sam", "Alex"]
Define def excited(name): so it returns "name!"
Make an empty list before the loop: excited_names = []
Write a for loop over names that:
calls excited(name)
appends the result to excited_names
After the loop, print(excited_names)

Run it, then Submit code so I can review it 😉

Python

```
names = ["Ron", "Sam", "Alex"]
def excited(name):
    return(name + "s")
excited_names = []
for name in names:
    excited_names.append(excited(name))
print(excited_names)
```

Nice, Ron — your structure is perfect 🎯

You:

Defined a per-item helper:
Python

```
def excited(name):
    return(name + "s")
```

Used a loop to delegate work and build a new list:
Python

```
excited_names = []
for name in names:
    excited_names.append(excited(name))
```

That’s exactly the pattern we want.

Only tiny thing: you chose "s" instead of "!", so you got ['Rons', 'Sams', 'Alexs']. If you change the helper to:

Python

```
def excited(name):
    return name + "!"
```

you’ll get ['Ron!', 'Sam!', 'Alex!'] 🎉

Now that you’ve seen both:

helper that prints inside the loop (scores)

helper that returns a value and you collect it in a list (names)

Do you feel clear on the difference between:

“just print the result” vs.
“return the result so I can store/use it later”?

You can answer: “yes, clear” or “not really yet”.

return S "name~"
thtrough e looks like S

Gotcha, I think I see what you were aiming for 👀

Right now you have:

Python

```
def excited(name):
    return name + "s"
```

If you want to return "name!", the pattern is:

Python

```
def excited(name):
    return name + "!"
```

So:

name is the input (like "Ron")
"!" is just a normal string stuck on the end
return name + "!" gives back "Ron!"
Quick check, just to be sure we’re on the same page:

In your own words, what’s the one job of excited(name) in this version?

(You can keep it very short, like: “take a name and …”)

tack on ! to the nae apparantly

Exactly 💯

The job of excited(name) is: take a name and tack on ! to it.

That’s a perfect “per-item helper” description.

Now, big picture for this pattern:

The loop: walks through the collection (names, scores, etc.)
The helper function: knows what to do with one item
Together: clean, reusable code that’s easy to read and change

To lock this in, want to do one last quick practice with a different helper (maybe uppercasing names or adding a prefix like "Dr. "), or do you feel solid with the pattern already?

Let's do another practice

Nice, let’s level it up a tiny bit 💪

We'll practice the same pattern but with a small twist: add a prefix to each name.

Goal:

Start with:
names = ["Ron", "Sam", "Alex"]
Write a helper that turns "Ron" into "Dr. Ron"
Use a loop + list building to end with:
["Dr. Ron", "Dr. Sam", "Dr. Alex"]

Step-by-step shape:

def add_title(name): → returns "Dr. " + name
names = [...]
titled_names = []
for name in names:
 call add_title(name)
 append the result to titled_names
print(titled_names) after the loop
Your turn in the code editor

Please:

Implement add_title(name) as described
Build titled_names using a for loop and the helper
Print titled_names

Run it, then hit Submit code so I can take a look.

Python

```
names = ["Ron", "Sam", "Alex"]
def add_title(name):
    return("Dr. " + name)
titled_names = []
for name in names:
    titled_names.append(add_title(name))
print(titled_names)
```

Beautifully done, Ron

You nailed the full pattern:

Python

```
def add_title(name):
    return("Dr. " + name)

titled_names = []
for name in names:
    titled_names.append(add_title(name))
```

Clear separation:

Helper function: handles one name → returns "Dr. " + name
Loop: walks through names and delegates work to the helper for each item
New list: titled_names collects all returned values

That's exactly what this lesson wants you to master ☑

One last tiny check and then we can wrap this lesson:

Which part of this loop is the place where the work is delegated to the helper function

— the “do this per item” step?

for name in names:
titled_names.append(add_title(name))
print(titled_names)

Just answer with 1, 2, or 3.

titled_names.append(add_title(name)) , the add_titke(name) part

Exactly right 🎯

titled_names.append(add_title(name)) is where the loop delegates the work to add_title for each item.

for name in names: → just walks through the list
add_title(name) → does the per-item work
append(...) → stores the result

You’ve shown you can:

Write a helper that processes one item
Call it inside a loop for each element
Collect the results in a new list

That’s the core mastery outcome for this lesson ☑

You’ve got this pattern down, Ron 💪

Are you ready to wrap up this lesson and move on to the next one?

Yes, let's move on

Nice work on this one, Ron 🎉

Quick recap of what you can now do:

Write a helper function that handles one item (like add_title(name) or excited(name)).
Use a for loop to walk through a list.
Inside the loop, delegate the work to the helper for each element.
Either:
just print the helper’s result, or
collect all results into a new list (like titled_names).
This “loop + helper” pattern is a big building block for clean, real-world code. You used it with scores and names — the same idea works for prices, messages, anything in a list.

Whenever you’re ready, you can jump into the next lesson in the Learn tab.
If anything from this one feels fuzzy later, just tell me what part and we’ll sharpen it up together 🔧🙂

```
names = ["Ron", "Sam", "Alex"]
def excited(name):
    return(name + "!")
excited_names = []
for name in names:
    excited_names.append(excited(name))
print(excited_names)

names = ["Ron", "Sam", "Alex"]
def add_title(name):
    return("Dr. " + name)
titled_names = []
for name in names:
    titled_names.append(add_title(name))
print(titled_names)
```